Iranian Journal of Science and Technology, Transactions of Electrical Engineering
https://doi.org/10.1007/s40998-019-00234-2

RESEARCH PAPER

Metastability-Induced TRNG Architecture on FPGA

R. Sivaraman[1] · Sundararaman Rajagopalan[1] · A. Sridevi[1] · J. B. B. Rayappan[1] · Moorthi Paramasivam Vijaya Annamalai[2] · Amirtharajan Rengarajan[1]

Received: 2 November 2018 / Accepted: 22 June 2019
© Shiraz University 2019

Abstract

Due to the emerging high-speed digital infrastructure, the protection of data being shared throughout the open networks has been a challenging task. By large, the dependency on cryptographic primitives exists to encounter cyberspace challenges. Key generation is a core process of any cryptographic application which improvises the strength of the algorithm. Reconfigurable hardware-assisted true random number generators (TRNGs) play a crucial role in key generation to provide high-speed cryptographic solutions. In this work, metastability-influenced TRNG architecture on Altera Cyclone II EP2C20F484C7 FPGA has been proposed. The 256 units of SR latches with de-synchronisation technique were the prime source used to harvest the true randomness. TRNG design consumed 1851 logic elements with a dynamic power dissipation of 4.41 mW. Proposed architecture achieves a high throughput of 26.64065 Mbps using 27 MHz onboard sampling clock. This TRNG has been validated through entropy, correlation, NIST SP 800-22 batteries of test, linear complexity test, restart experiment and hamming distance analysis.

Keywords TRNG · Metastability · FPGA · Cryptography

1 Introduction

Growing advancements in communication necessitate the enhancement of information security to accomplish the CIA (confidentiality, integrity and authentication/availability) triad. The backbone of such security implementation is key generation which is employed in many applications but not limited to initialization vector (IV) generation, nonces, one-time passwords (OTP), protocol authentication, random simulation, gaming, digital signatures, etc. In general, keys must have three fundamental properties namely (1) Uniform distribution of zeros and ones; (2) They should not be predictable; and (3) They must not produce any pattern (Sunar 2009). Keys are random sequences generated through random number generators (RNGs). RNGs are the basic security primitives which are classified into two types, namely pseudo-random number generators (PRNGs) and TRNGs. PRNGs can be realised through a mathematical equation or a fixed architecture in which the random sequences are generated by providing an initial condition and/or a start-up seed. Cellular automata (Tomassini and Perrenoud 2001), chaotic maps such as logistic map (Ismail et al. 2018), tent map (Martínez-Ñonthe et al. 2012), and piece-wise linear map (PWLCM) (Ramalingam et al. 2017), chaotic attractors namely Lorenz (Lorenz 1963), Lu (Liu et al. 2004) are few examples of PRNG. Though PRNG produces random numbers, the nature of randomness is deterministic. Due to the degree of security and unpredictability, TRNGs are required widely in real-time applications (Gilli et al. 2011).

Most of the TRNG architectures proposed so far rely upon on software and/or hardware approaches. In particular, FPGA-based TRNG solutions have been proposed during recent years (Elbirt et al. 2001). Due to the advantages namely Reconfigurability, Reusability, Algorithm agility, Easy prototyping, Parallelism, Architecture efficiency, Hard-wired capability, Faster time to market and High throughput, FPGA has become an apt platform for cryptography. Thermal noise in electronic circuits (Jun and Kocher 1999), clock jitters produced through ring oscillator

✉ Amirtharajan Rengarajan
amir@ece.sastra.edu

[1] School of EEE, SASTRA Deemed University, Thanjavur, Tamilnadu, India

[2] Cyber Security and Risk Services, Wipro Technologies Ltd., Bangalore, India

Published online: 01 July 2019

(RO) (Kohlbrenner and Gaj 2004), phase-locked loop (PLL) and delay-locked loop (DLL) (Fischer and Drutarovský 2003; Fischer et al. 2004; Kwok and Lam 2006), digital clock managers (DCM) (Johnson et al. 2017), block random access memory (BRAM) write collisions (Güneysu and Paar 2009), Boolean chaotic oscillators (Park et al. 2015) and flip-flop's metastability (Danger et al. 2007) have been considered as the sources of true randomness in FPGA in various works. Among all the sources, RO and flip-flop's metastability have been chosen as eminent True random sources which offer device independence and a high degree of accumulation of true randomness.

Initially, clock jitters-based TRNG architecture was proposed by Tsoi et al. (2003), wherein a high-frequency clock was sampled using an unstable low-frequency clock to generate true random bits. High-frequency clock was driven through an external oscillator, and the design also utilised BBS (Blum Blum Shub) generator for processing the true random bits. The design was implemented on Xilinx Virtex XCV300E FPGA at a throughput of 225 bps. Free-running oscillator designs have also been suggested for true random number generation (Dichtl and Golić 2007).

Generation of true random numbers through jittery clocks was elaborately exploited by Sunar et al. (2007) where phase jitter of clocks has been taken as entropy source. This study of RO produces a result when the number of oscillators is exponentially increased to achieve a constant fill rate. This work also depicts a "coupon collector" problem when identical inverter rings were used for constructing the TRNG. This theoretical modelling included an RO constructed using 330 inverters, XOR tree and a resilient function as a post-processing unit which yielded an entropy of 0.97.

In this context, Schellekens et al. continued the work of Sunar et al. on the FPGA platform (Schellekens et al. 2006). Three versions of RO-based TRNG were illustrated in this work. Initially, TRNG proposed in Sunar et al. (2007) was implemented on Xilinx Virtex XC2VP30 FPGA which consumed 1664 slices. The modified version of this TRNG was designed using 110 rings of ROs with each ring had three inverters and while realisation on FPGA, it occupied 565 slices. Finally, 210 rings of RO design were suggested to yield good entropy results having utilised 973 slices. This work yielded a throughput of more than 2 Mbps at a sampling frequency of 40 MHz.

Wold and Chik enhanced the RO-based TRNG presented in Sunar et al. (2007) further using Altera Cyclone II FPGA (Wold and Tan 2008). The enhancement was carried out on sampling with an additional D Flip-flop at each result of RO rings to avoid the high transition effects of RO and metastability caused in D Flip-flop. In this architecture, two versions of TRNG were designed in which the former one with 25 rings of ROs consumed 83 logic elements and the other being 50 rings of ROs utilised 167 logic elements. Restart experiment had also been conducted to prove the true randomness of enhanced TRNG.

Also, to reduce the number of inverters, special case of RO called transition effect RO had been suggested which captured the intrinsic noise in the FPGA to produce random bits (Varchola and Drutarovsky 2010). Though RO-based TRNG produces good results, it suffers from coupon collector problem which was discussed by Sunar et al. (2007). As a result, it is necessary to incorporate nearly 600 inverters to yield the required randomness which consumes more area and power in the design architecture.

Hence, the next ultimate solution of TRNG on FPGA is the exploitation of metastability on flip-flops. In general, D (Wieczorek 2013) and SR (Set–Reset) latches are preferred for building TRNG architectures. Hisashi and Shuichi presented one such work, wherein the SR latches were constructed as hard macrocomponents using look-up tables (LUTs). Since the Xilinx Virtex 4 FPGA was considered as the target platform, two LUTs of a slice were utilised to form a single SR latch. Three types of LUT latch designs with 64, 128 and 256 SR latches were used to achieve 12.5 Mbps throughput (Hisashi and Ichikawa 2012). Continuation of this work was done by Torii et al. (2016) who XORed 256 SR latches in Xilinx Spartan FPGA, and the resultant signal was sampled to generate true random bits. Majzoobi et al. proposed a method of TRNG using metastability regulated by PLD (programmable delay line). These PLDs are effective delay adjusting components through which the delay is elapsed or reduced. When PLD is used in metastable circuits, it controls the operations of flip-flop such that whenever flip-flop reaches normal condition, PLD brings it to the metastable region to induce the random behaviour (Majzoobi et al. 2011).

From the literature, the metastability property of flip-flops would be more suitable for FPGA-based TRNG designs due to their distinct features. This proposed work also concentrates on metastability to provide one more solution to generate true random numbers through SR latches and LFSR (linear feedback shift register). The merits of proposed TRNG are as follows:

- No keeper circuit is required to hold the metastable data from the entropy source
- No compression function has been employed for post-processing
- The lightweight post-processing technique is utilised to improve the statistical properties of random numbers
- Device-independent TRNG architecture which suits any FPGA family

- The throughput of the design is relatively high in comparison with other earlier works
- The TRNG did not use clock synthesisers and PLLs
- Area-optimised entropy source which requires only 865 logic elements out of 18,752 (4.6%) to generate true randomness

2 Proposed Methodology

A TRNG consists of three basic building blocks namely entropy source, harvesting mechanism and post-processing. Generic architecture of TRNG is illustrated in Fig. 1 in which the entropy source is the source of true randomness evolving from any physical mean. This true randomness is further enhanced using a harvesting mechanism governed through de-synchronisation or jitter counter techniques. When a single entropy source is used, the jitter counting method is adopted (Kohlbrenner and Gaj 2004), whereas the de-synchronization is chosen for multi-event entropies (Schellekens et al. 2006). Finally, the sampling operation is carried out to accumulate the true randomness to form random bits. In general, TRNG produces unpredictable random numbers with poor statistical properties. Hence, post-processing is an additional circuitry mostly based on compression function used to enhance the results of TRNG to make it statistically strong.

The proposed TRNG has a collection of SR latches as an entropy source. It consists of 256 units of SR latches, logical XOR with D flip-flop as harvesting mechanism and LFSR employed for post-processing. Figure 2 depicts the functional block diagram of the proposed TRNG. It includes a sampling unit and storage unit in addition to the basic building blocks of TRNG.

2.1 Metastability on SR latches

In this work, the SR latch has a crucial role in inducing metastability. Structure of the SR latch includes two NAND gates, and the outputs are cross-coupled to hold the data as shown in Fig. 3. In a general scenario, inputting the S and R simultaneously is not advisable. However, to create a metastable state, it is necessary to keep the SR latch at the metastable region, thereby providing the same input logic levels to S and R simultaneously. If the input signal to S and R is a clock pulse, it would be more efficient to attain the state of metastability very easily. A time-varying feature of the clock signal results in more switching activity in the latch with more uncertainty. In this work, 256 SR latch units were designed using 512 NAND gates clocked through a pulse of 50 MHz onboard clocks.

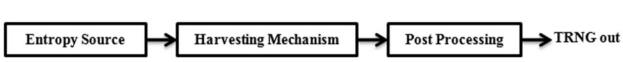

Fig. 1 Generic TRNG architecture

In Fig. 3, when the clock signal Clk is on a negative edge, the output of S and R become {Q, QB} = {1, 1}. When the clock edge shifts from negative to positive, the latch try to become stable at {0, 1} or {1, 0} which introduces the metastability, this operation is continued to harvest the metastabilities in all the registers through the sampling process at a frequency of 27 MHz. In this work, for harvesting the metastability XOR structure is preferred for fetching the true randomness from the entropy source. Let $\{L_1, L_2, \ldots. L_{256}\}$ be the output bits from the SR latches. A simple, lightweight logical function is required to generate a diffused bit representation of individual metastability from the latches. Hence, $f(L_1, L_2, \ldots L_{256}) = L_1$ (*) L_2 (*) ... (*) L_{256}, where (*) may be AND, OR, NAND, NOR, XOR or XNOR function. Considering the functional characteristic and utility in RNGs, XOR has been chosen for harvesting the randomness owing to its complete dependency and sensitivity to variations on the status of entire latches.

2.2 Post-Processing

Post-processing is an enhancement unit for any TRNG. It will improvise the statistical properties of TRNG in terms of equidistribution of zeros and ones. LFSR has been proposed as a post-processor in earlier works (Schulz 2004; Fischer and Bernard 2011). It is utilised in this work to post-process the raw true random bits to yield high entropy.

LFSR is constructed based on the primitive polynomials. LFSR employs a collection of registers to shift and hold the data concerning the synchronous clock. A 128-bit LFSR has been used in this work whose primitive polynomial is expressed as $X^{128} + X^{127} + X^{126} + X^{121} + 1$. Block diagram of this LFSR is shown in Fig. 4. Since LFSR is a pseudo-random sequence generator, the initial seed is required to trigger it. Further, post-processed random bits were sampled at 27 MHz frequency and stored in on-chip RAM of size 128 $\times$ 1024 consists of 131,072-bit locations. The accumulation of 32-bit TRNG on BRAM memory captured through In-System Memory Content Editor is shown in Fig. 5, and the time series evolution of random number are depicted in Fig. 6.

3 Results and Discussion

Proposed metastability-based TRNG was developed using VHDL and implemented on Altera Cyclone II EP2C20F484C7 FPGA using Quartus II 8.0 EDA

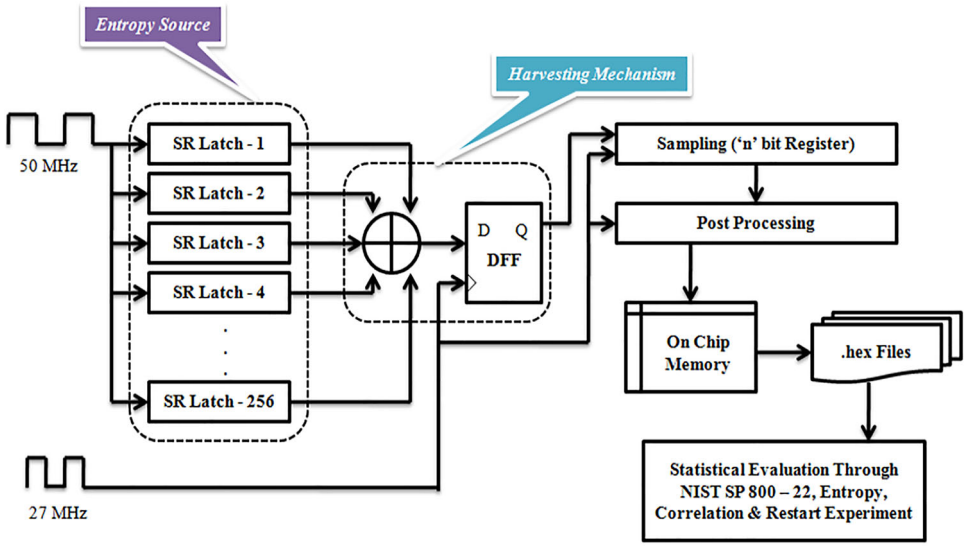

Fig. 2 Block diagram of proposed TRNG

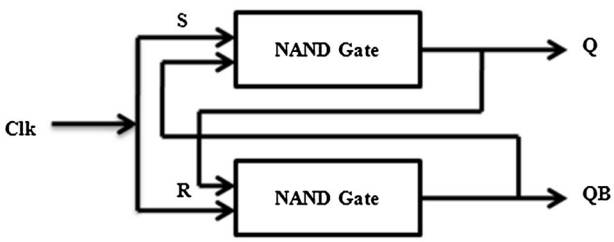

Fig. 3 Representation of SR latch

(Electronic Design Automation Tool). True random numbers were generated in the widths of 128, 64, 32 and 16 bits on FPGA and further stored in on-chip memory. SR latch-based TRNG without post-processing consumes 865 configurable logic blocks which are 4.6% of total logic elements with a total power dissipation of 72.64 mW. To verify the statistical properties of TRNG, Entropy, Correlation, TestU01-Linear Complexity and NIST SP 800-22 batteries of test analyses were performed. True randomness of the proposed TRNG was validated through restart experiment and hamming distance analysis. Also, hardware and power dissipation have also been reported.

3.1 Entropy Analysis

Entropy is a measure of randomness which defines the probability of equidistribution of zeros and ones among the set of random numbers (Wu et al. 2013). It is a preliminary test to ensure the equidistribution which can lead to further exploration of TRNG statistics. Shannon entropy can be defined as,

$$H(m) = -\sum_{i=1}^{N} P(m_i)\log_2 P(m_i) \tag{1}$$

where $P(m_i)$ is the probability of appearance of the data m_i and m_i may either be 0 or 1.

For a strong random number generator, the entropy value must be close to 1. To analyse the equidistribution property, entropy analyses have been conducted for 128-, 64-, 32- and 16-bit random numbers and the results are presented in Table 1. It was inferred that the obtained values were close to 1 encouraging further analyses.

3.2 Correlation Analysis

Correlation is one important analysis which evaluates the trace of the relationship between the successive random

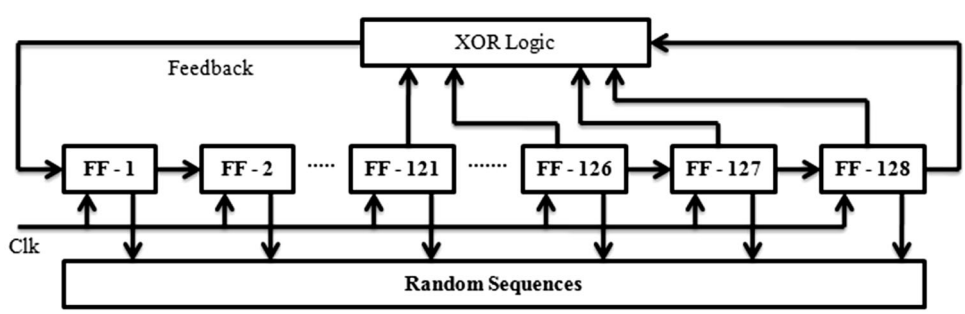

Fig. 4 LFSR architecture-128 bit

Fig. 5 32-bit TRNG on BRAM

C4	1F	E5	F5	B0	A5	89	2C	60	A3	49	98
17	9C	F1	7B	49	9C	51	53	77	39	A0	73
02	A2	D9	D0	0D	A8	A1	38	06	28	B3	C1
16	7F	4B	EA	14	DF	99	D4	64	DD	19	48
34	BF	BA	F8	60	BE	1A	F5	AA	21	56	C3
69	95	B8	6F	15	B8	35	77	D7	39	41	77
7D	5C	32	E7	FF	D4	65	E7	DC	35	6A	4F
E5	EE	B8	ED	CD	17	AF	04	8F	CF	AD	EF
D9	DC	32	B1	59	40	32	28	9D	DF	12	A8
B8	06	7D	1A	AB	AE	68	1A	DB	AD	B8	0E
43	3A	DD	BE	92	00	97	76	EE	88	85	D6
15	A6	5E	F9	4A	B0	5A	41	54	8E	72	B9
FF	43	4F	E6	FE	4C	C7	DA	2E	46	47	EE
4A	E1	21	B9	16	E3	C1	BE	16	BB	D7	3E

Fig. 6 Time series evolution of true random numbers

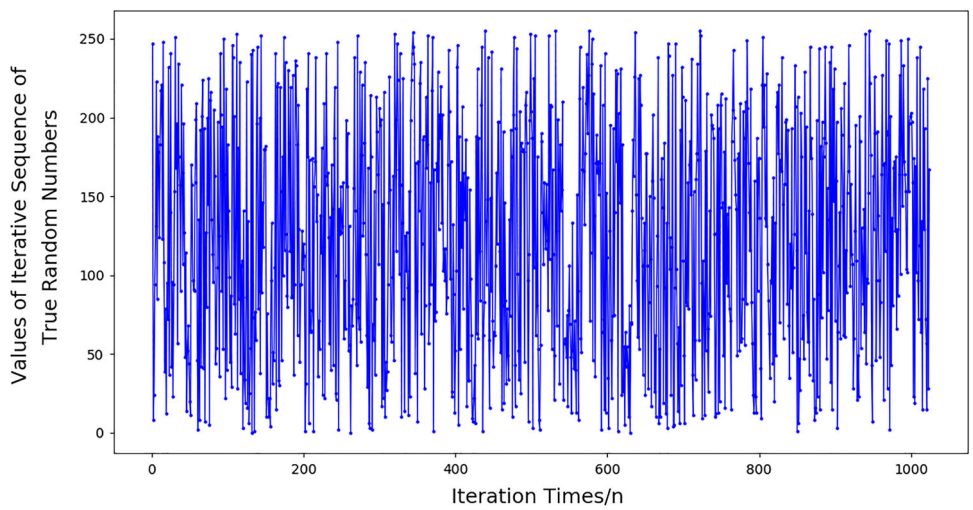

Table 1 Entropy analysis

TRNG sets	SR latch-based TRNG without post-processing				SR latch-based TRNG with post-processing			
	128 bit	64 bit	32 bit	16 bit	128 bit	64 bit	32 bit	16 bit
Set-1	0.9777422	0.958518	0.916402	0.965397	0.999987	0.999996	0.999980	0.999995
Set-2	0.977375	0.958616	0.915969	0.966597	1	0.999976	0.999981	0.999953
Set-3	0.977226	0.958422	0.915535	0.967245	0.999998	0.999999	0.999988	0.999941
Set-4	0.977202	0.958711	0.917419	0.96473	0.999999	0.999899	0.999976	0.999944
Set-5	0.977238	0.958593	0.915814	0.966979	0.999991	0.999992	0.999966	0.999999

numbers which is vital to justify the strength and unpredictability. To become a cryptographically secure TRNG, the correlation between the random numbers must be close to zero. Extensive analysis was performed for 128-bit true random numbers, and the correlation coefficients obtained were − 0.0070, 0.0087 and 0.0229 in horizontal, vertical and diagonal directions. The random distribution of the generated bits is expressed through a scatter plot as shown in Fig. 7.

3.3 NIST SP 800-22 Tests

Table 2 presents the results of NIST SP 800-22 tests to evidence the statistical efficiency of the TRNG. It is a test suite provided by National Institute of Standards and Technology (NIST) which comprise several tests to evaluate the selection of random number generators and testing. The batteries of test examine the statistical properties of given random sequences in terms of randomness through

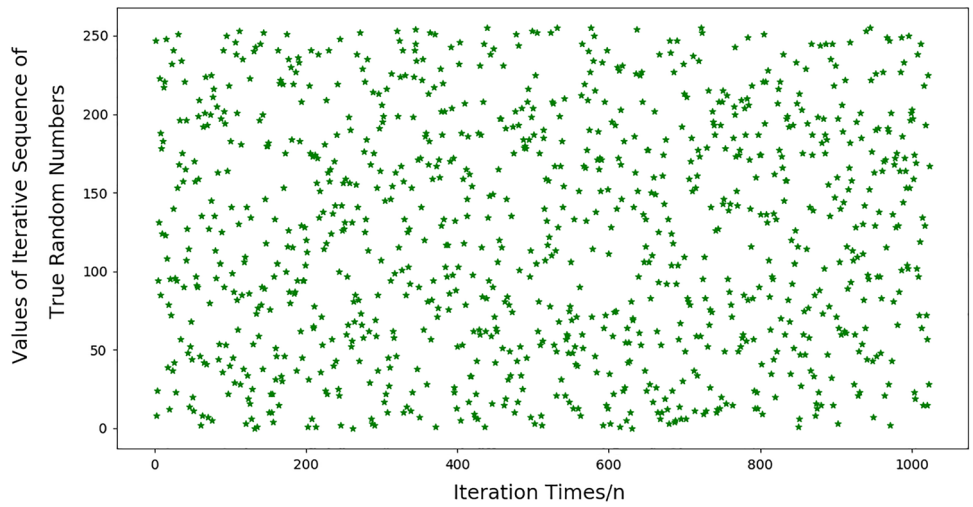

Fig. 7 Scatter plot of true random numbers

the probability distribution and unpredictability. The probability values (P) in the table may vary between 0 and 1 which are calculated for each of the 15 NIST Tests. Very low P values indicate the absence of randomness in the stream of bits. NIST SP 800-22 standard (Bassham et al. 2010) observes that if the P value is greater than or equal to 0.001, then the bits under test can be considered to have uniform distribution and statistical strength.

In this work, all the 128-, 64-, 32- and 16-bit TRNGs have been tested using NIST test suite and the proportion (P) values of five iterations of 128-bit TRNG are listed in Table 2. This analysis was conducted with 131,072 true random bits. The following parameter settings were applied while performing NIST randomness analyses.

Block frequency test: Block length (M) = 128
Non-overlapping template: Block length (m) = 9
Overlapping template: Block length (m) = 9
Approximate entropy test: Block length (m) = 8
Serial test: Block length (m) = 10
Linear complexity test: Block length (M) = 500

3.4 Linear Complexity Test-TestU01

Purpose of this test is to identify whether the sequence is complex enough to be considered as random or non-random. In this test, random sequences are characterised by the polynomials of LFSR. It performs jump complexity and jump size tests on the random numbers to determine its linear complexity. For each z with $1 \leq z \leq n$, the linear complexity of the first z-bits of any sequence can be

Table 2 NIST SP 800-22 batteries of tests

Test parameters	P values for five iterations				
	I1	I2	I3	I4	I5
Frequency	0.350485	0.213309	0.739918	0.911413	0.534146
Block frequency	0.350485	0.739918	0.350485	0.534146	0.534146
Cumulative sums-I	0.911413	0.122325	0.122325	0.739918	0.911413
Cumulative sums-II	0.911413	0.008879	0.213309	0.739918	0.122325
Runs	0.213309	0.213309	0.213309	0.122325	0.350485
Longest run	0.739918	0.350485	0.534146	0.739918	0.534146
Rank	0.739918	0.911413	0.739918	0.122325	0.350485
FFT	0.017912	0.350485	0.350485	0.534146	0.534146
Non-overlapping template	0.911413	0.911413	0.911413	0.911413	0.911413
Overlapping template	0.739918	0.739918	0.213309	0.350485	0.122325
Approximate entropy	0.739918	0.350485	0.739918	0.122325	0.066882
Serial-I	0.534146	0.739918	0.534146	0.035174	0.739918
Serial-II	0.035174	0.350485	0.122325	0.534146	0.350485
Linear complexity	0.739918	0.213309	0.534146	0.122325	0.534146

Table 3 Results of linear complexity test-TestU01

Test with functional parameters	*P* value		Result
Linear complexity (N = 1, n = 25,600,000 bits, r = 0, s = 1)	Normal statistic	0.88	Pass
	Chi-square statistics	0.73	
Linear complexity (N = 1, n = 25,600,000 bits, r = 29, s = 1)	Normal statistic	0.9905	Pass
	Chi-square statistics	0.18	

computed through Berlekamp–Massey algorithm (L'Écuyer and Simard 2002). Table 3 presents the result of linear complexity test from TestU01 test suite which ensures that the random numbers generated from the proposed TRNG do not have any dependency with good complexity. The test was performed for two different values of r namely 0 and 29 as given in the BigCrush battery of tests while keeping the remaining parameters unchanged. The obtained P values for the normal statistic were 0.88 and 0.9905, respectively, for these two cases.

3.5 Restart Experiment

Restart experiment is a fundamental analysis to differentiate true randomness and pseudo-randomness (Wold and Tan 2008; Martin et al. 2016). TRNG generates new random sequences during each iteration, whereas the PRNG generates the same set of random sequences for all the iterations under the same seed. Restart experiment has been conducted on raw true random bits for 50 iterations, and Fig. 8 illustrates the obtained results of five iterations

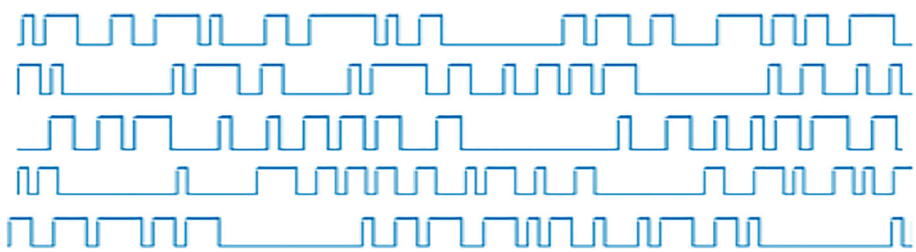

Fig. 8 Results of restart experiment

Table 4 Average hamming distance calculation

TRNG sets	SR latch-based TRNG with post-processing			
	128 bit	64 bit	32 bit	16 bit
Set-1	65.1533203125	44.832411324	16.068359375	11.5986328125
Set-2	65.2476407419	43.132310718	16.092417832	11.6296778392
Set-3	65.2215714982	41.907085691	16.114527411	11.6412884333
Set-4	65.1991020886	41.174951203	16.129806753	11.6439586179
Set-5	65.1746909564	40.649379618	16.129806753	11.6496421600

Table 5 Hardware analysis

Parameters (Target Platform-Altera Cyclone II EP2C20F484C7 FPGA)	128-bit TRNG	64-bit TRNG	32-bit TRNG	16-bit TRNG
Logic elements consumption				
Combinational functions	1851	1509	1351	1273
Logic registers	1316	1005	844	763
Power dissipation (mW)				
Static power	47.36	47.36	47.36	47.35
Dynamic power	4.41	2.32	1.70	1.00
IO power	20.87	20.87	20.87	20.87
Memory bits (no. of keys = 1024)	131,072	65,536	32,768	16,384

Table 6 Hardware consumption comparison

Size of RNG	Target FPGA	Reference	Hardware consumption
8-bits	Xilinx Virtex XCV300E-8	Tsoi et al. (2003)	310 slices
8-bits	Altera EP20K200E	Fischer and Drutarovsky (2003)	121 logic cells with 4 embedded system blocks
8-bit	Altera EP1S25	Fischer et al. (2004)	120 logic elements
128-bits	Xilinx Virtex II Pro Board	Kwok and Lam (2006)	486 slices
Sequence of bits (single bit at a time)	Xilinx Virtex XCV1000	Kohlbrenner and Gaj (2004)	1 CLB (configurable logic block)
NA	Xilinx Virtex XC2VP30	Schellekens et al. (2006)	973 slices
Sequence of bits (single bit at a time)	Altera EP1S25	Danger et al. (2007)	10 LAB
NA	Xilinx Virtex XC4VFX20	Hisashi and Shuichi (2012)	580 slices
128-bit	Altera Cyclone II EP2C20F484C7 FPGA	Proposed work	1851 logic elements

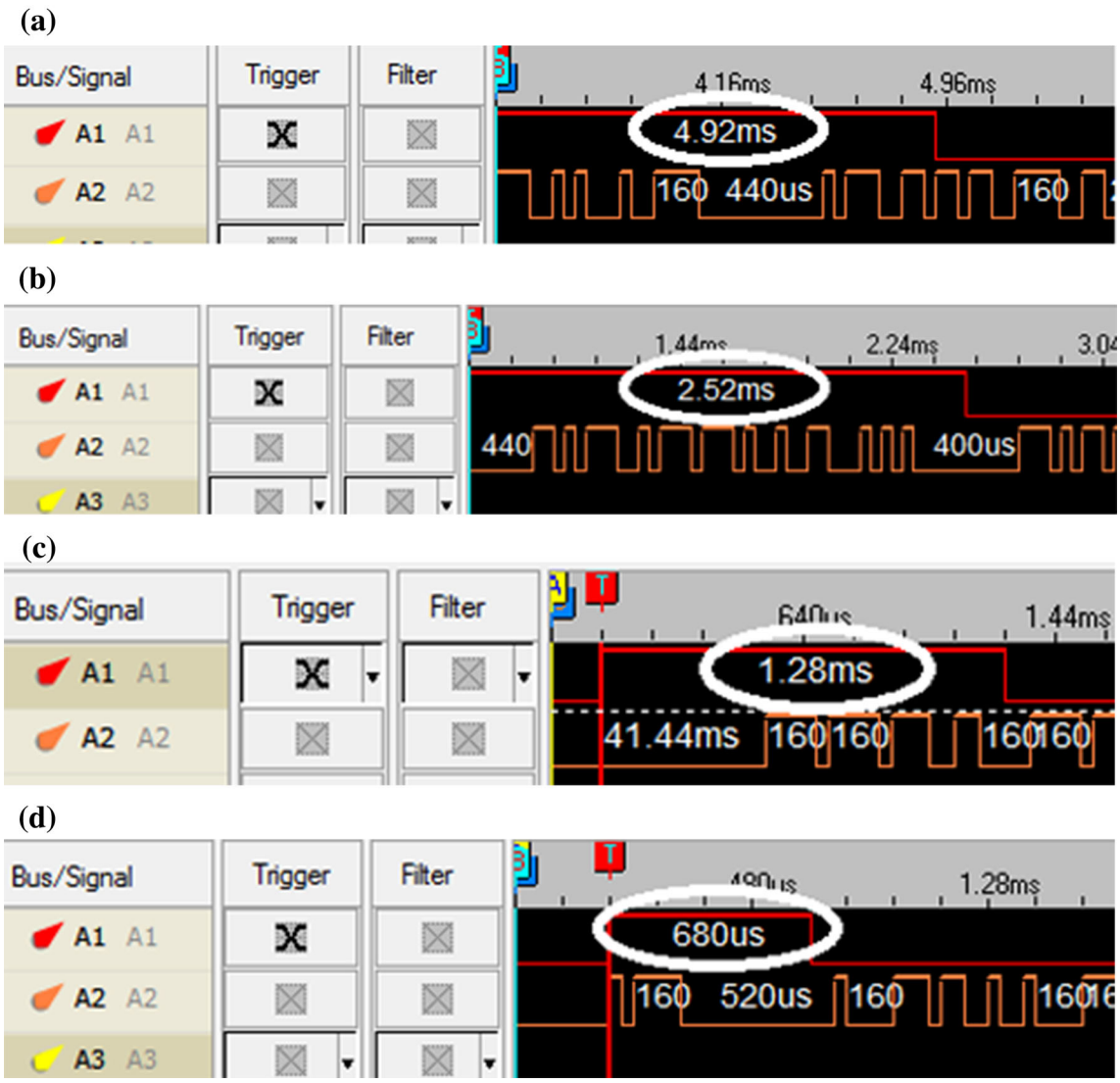

Fig. 9 Timing analysis: **a** 128-bit TRNG; **b** 64-bit TRNG; **c** 32-bit TRNG and **d** 16-bit TRNG

Table 7 Throughput analysis

Parameters	128-bit TRNG	64-bit TRNG	32-bit TRNG	16-bit TRNG
Time	4.92 mS	2.52 mS	1.28 mS	680 µS
Throughput (Mbps)	26.64065	26.00634	25.60000	24.094111

acquired using the external logic analyser. The inference clarifies the presence of true randomness in the proposed work.

3.6 Hamming Distance Analysis

Hamming distance is one another test to evidence the true randomness by observing the transitions of zeros and ones between two true random numbers. The hamming distance $d(x, y)$ between any two vectors $x, y \in F^{(n)}$ is the number of coefficients in which they differ (Pieterse and Black 2001). The hamming distance can be computed as follows:

$$\mathrm{HD}\ (x_1, x_2 \ldots x_n) = f(x_i(x_{i1}, x_{i2}, \ldots x_{iM}) \oplus x_{i+1}(x_{(i+1)1}, x_{(i+1)2}, \ldots x_{(i+1)M})) \quad (2)$$

where $0 < i < N$. Here, x_1, $x_2 \ldots$ x_n are the random values, and function f computes the sum of the bits in resulting XOR operation between x_i and x_{i+1}. To analyse the effect of true randomness, hamming distance has been computed between all the true random numbers of five different sets and the average values of hamming distance are presented in Table 4.

From the table, it was inferred that close to 50% of logic level transitions resulted during each iteration of TRNG which is possible when the existence of 0's and 1's remain almost equal thereby avoiding bias towards a specific bit. This transition count form of hamming distance is a good measure of true randomness.

3.7 Hardware Analysis

It is so important to compute the consumption of logic elements and power dissipation for any digital design being implemented on FPGA. Logic elements are the combination of LUTs, registers, multiplexers, latches and dedicated carry chain logics which are essential to accommodate the required design. Detailed hardware analyses have been performed for the proposed TRNG, and the results are shown in Table 5 and the comparison of proposed TRNG architecture with the earlier TRNG designs is shown in Table 6.

3.8 Throughput Analysis

Time taken for generating the true random numbers was estimated through Zero Plus logic analyser, and Fig. 9a–d depicts the timing analysis of the proposed TRNG. To employ the TRNGs in real-time systems, high speed is a major concern. This proposed work has achieved a throughput of 26.64065 Mbps with 27 MHz of the sampling clock to generate 131,072 true random bits (Table 7). Throughput of the proposed work has been compared with the other existing techniques of TRNG which is shown in Table 8.

Table 8 Throughput comparison

Entropy source	Target FPGA	Reference	Throughput (Mbps)
External oscillator	Xilinx Virtex XCV300E-8	Tsoi et al. (2003)	0.03
DLL/PLL	Altera EP20K200E	Fischer and Drutarovsky (2003)	0.07
	Altera EP1S25	Fischer et al. (2004)	1.00
	Xilinx Virtex II Pro Board	Kwok and Lam (2006)	6.00
Ring oscillators	Xilinx Virtex XCV1000	Kohlbrenner and Gaj (2004)	0.5
	Xilinx Virtex XC2VP30	Schellekens et al. (2006)	2.5
	Xilinx XC3S200	Dichtl and Golic (2007)	12.5
Transition effect RO	Xilinx Spartan 3E Board	Varchola and Drutarovsky (2010)	0.25
D latch metastability	Altera EP1S25	Danger et al. (2007)	20
RS latch metastability (64 LUT latch)	Xilinx Virtex XC4VFX20	Hisashi and Shuichi (2012)	3.85
RS latch metastability (128 LUT latch)	Xilinx Virtex XC4VFX20		8.33
RS latch metastability (256 LUT latch)	Xilinx Virtex XC4VFX20		12.5
RS latch metastability + LFSR	Altera Cyclone II EP2C20F484C7 FPGA	Proposed work	26.64065

4 Conclusion and Future Work

Multi-event metastability on SR latches with LFSR post-processing has been presented in this work for generating the true random numbers. This architecture took 256 units of SR latches which was implemented on Altera Cyclone II EP2C20F484C7 FPGA. The onboard clock 50 MHz has been used as input to the latches wherein 27 MHz was utilised for sampling the true random bits. The proposed design has been evaluated through statistical analyses such as entropy and correlation. Verification of randomness has been performed using NIST SP 800-22 test suite and linear complexity test of Test U01. Further, restart experiment was also conducted to evidence the true randomness. Extensive hardware analyses have been carried out to prove the efficiency on the FPGA. The suggested TRNG can be embedded in cryptoprocessors as a reliable key generation unit. Future work will be on experimenting the proposed design with jitter clocks generated through RO of identical and non-identical rings.

Acknowledgements The authors wish to thank SASTRA Deemed University for providing the infrastructure through the Research & Modernization Fund (Ref. No. R&M/0026/SEEE-010/2012-13) to carry out the research work.

References

Bassham LE, Rukhin AL, Soto J, Nechvatal JR, Smid ME, Barker EB, Leigh SD, Levenson M, Vangel M, Banks DL, Heckert NA, Dray JF, Vo S (2010) A statistical test suite for random and pseudorandom number generators for cryptographic applications. National Institute of Standards and Technology, Gaithersburg, MD, USA, Technical Report. https://doi.org/10.6028/nist.sp.800-22r1a

Danger JL, Guilley S, Hoogvorst P (2007) Fast true random generator in FPGAs. In: 2007 IEEE North-East workshops on circuits and systems, NEWCAS 2007, pp 506–509. https://doi.org/10.1109/newcas.2007.4487970

Dichtl M, Golić JD (2007) High-speed true random number generation with logic gates only. In: Paillier P, Verbauwhede I (eds) Cryptographic hardware and embedded systems, CHES 2007. Springer, Berlin, pp 45–62

Elbirt AJ, Yip W, Chetwynd B, Paar C (2001) An FPGA-based performance evaluation of the AES block cipher candidate algorithm finalists. IEEE Trans Very Large Scale Integr Syst 9:545–557. https://doi.org/10.1109/92.931230

Fischer V, Bernard F (2011) True random number generators in FPGAs. In: Badrignans B, Danger JL, Fischer V, Gogniat G, Torres L (eds) Security trends for FPGAS. Springer, Dordrecht

Fischer V, Drutarovský M (2003) True random number generator embedded in reconfigurable hardware. In: Kaliski BS, Koç ÇK, Paar C (eds) Cryptographic hardware and embedded systems, CHES 2002. 4th international workshop, Redwood Shores, CA, USA, August 13–15, 2002 revised paper. Springer, Berlin, pp 415–430. https://doi.org/10.1007/3-540-36400-5_30

Fischer V, Drutarovský M, Šimka M, Bochard N (2004) High performance true random number generator in Altera Stratix FPLDs. In: Becker J, Platzner M, Vernalde S (eds) Field programmable logic and application, FPL 2004, Leuven, Belgium, August 30–September 1, 2004. Proceedings. Springer, Berlin, pp 555–564. https://doi.org/10.1007/978-3-540-30117-2_57

Gilli M, Maringer D, Schumann E (2011) Chapter Six: Generating random numbers. In: Gilli M, Maringer D, Schumann E (eds) Numerical methods and optimization in finance. Academic Press, San Diego, pp 119–158. https://doi.org/10.1016/B978-0-12-375662-6.00006-7

Güneysu T, Paar C (2009) Transforming write collisions in block RAMs into security applications. In: Proceedings of 2009 international conference on field-programmable technology, FPT'09, pp 128–134. https://doi.org/10.1109/fpt.2009.5377631

Hisashi H, Ichikawa S (2012) FPGA implementation of metastability-based true random number generator. IEICE Trans Inf E95.D:426–436. https://doi.org/10.1587/transinf.e95.d.426

Ismail SM, Said LA, Radwan AG, Madian AH, Abu-Elyazeed MF (2018) Generalized double-humped logistic map-based medical image encryption. J Adv Res 10:85–98

Johnson AP, Chakraborty RS, Mukhopadyay D (2017) An improved DCM-based tunable true random number generator for Xilinx FPGA. IEEE Trans Circuits Syst II Express Briefs 64:452–456. https://doi.org/10.1109/TCSII.2016.2566262

Jun B, Kocher P (1999) The Intel random number generator. Cryptograph Research Inc., White Paper, vol 27, pp 1–8

Kohlbrenner P, Gaj K (2004) An embedded true random number generator for FPGAs. In: Proceedings of 2004 ACM/SIGDA 12th international symposium on field-programmable gate arrays. ACM, New York, pp 71–78. https://doi.org/10.1145/968280.968292

Kwok SHM, Lam EY (2006) FPGA-based high-speed true random number generator for cryptographic applications. In: 2006 IEEE regional 10 international conference, 2006, TENCON 2006, pp 1–4. https://doi.org/10.1109/tencon.2006.344013

L'Écuyer P, Simard R (2002) TestU01: a software library in ANSI C for empirical testing of random number generators

Liu C, Liu T, Liu L, Liu K (2004) A new chaotic attractor. Chaos Solitons Fractals 22(5):1031–1038

Lorenz EN (1963) Deterministic nonperiodic flow. J Atmos Sci 20:130–141

Majzoobi M, Koushanfar F, Devadas S (2011) FPGA-based true random number generation using circuit metastability with adaptive feedback control. Gastroenterology 101:17–32. https://doi.org/10.1007/978-3-642-23951-9_2

Martin H, Peris-Lopez P, Tapiador JE, San Millan E (2016) A new TRNG based on coherent sampling with self-timed rings. IEEE Trans Ind Inform 12:91–100. https://doi.org/10.1109/tii.2015.2502183

Martínez-Ñonthe JA, Castañeda-Solís A, Díaz-Méndez A, Cruz-Irisson M, Vázquez-Medina R (2012) Chaotic block cryptosystem using high precision approaches to tent map. Microelectron Eng 90:168–172

Park M, Rodgers JC, Lathrop DP (2015) True random number generation using CMOS Boolean chaotic oscillator. Microelectron J 46:1364–1370. https://doi.org/10.1016/j.mejo.2015.09.015

Pieterse V, Black P (2001) Hamming distance. In: MATH32031: coding theory, pp 706–706

Ramalingam B, Rengarajan A, Rayappan JBB (2017) Hybrid image crypto system for secure image communication—a VLSI approach. Microprocess Microsyst 50:1–13

Schellekens D, Preneel B, Verbauwhede I (2006) FPGA vendor agnostic true random number generator. In: Proceedings of 2006 international conference on field programmable logic and applications. FPL, pp 139–144. https://doi.org/10.1109/fpl.2006.311206

Schulz RA (2004) Random number generator circuit, US 6,807,553 B2 Patent

Sunar B (2009) True random number generators for cryptography. In: Koç ÇK (ed) Cryptographic engineering. Springer, New York. https://doi.org/10.1007/978-0-387-71817-0

Sunar B, Martin WJ, Stinson DR (2007) A provably secure true random number generator with built-in tolerance to active attacks. IEEE Trans Comput 56:109–119. https://doi.org/10.1109/TC.2007.250627

Tomassini M, Perrenoud M (2001) Cryptography with cellular automata. Appl Soft Comput 1(2):151–160

Torii N, Kokubo H, Yamamoto D, Itoh K, Takenaka M, Matsumoto T (2016) ASIC implementation of random number generators using SR latches and its evaluation. EURASIP J Inf Secur 2016:1–12. https://doi.org/10.1186/s13635-016-0036-1

Tsoi KH, Leung KH, Leong PHW (2003) Compact FPGA-based true and pseudo random number generators. In: Proceedings of IEEE symposium on FPGAs custom computing machines, 2003, pp 51–61. https://doi.org/10.1109/fpga.2003.1227241

Varchola M, Drutarovsky M (2010) New high entropy element for FPGA based true random number generators. In: Mangard S, Standaert F-X (eds) Cryptographic hardware and embedded systems, CHES 2010. 12th international workshop, Santa Barbara, USA, August 17–20, 2010. Proceedings. Springer, Berlin, pp 351–365. https://doi.org/10.1007/978-3-642-15031-9_24

Wieczorek PZ (2013) Dual-metastability FPGA-based true random number generator. Electron Lett 49:744–745. https://doi.org/10.1049/el.2012.4126

Wold K, Tan CH (2008) Analysis and enhancement of random number generator in FPGA based on oscillator rings. In: Proceedings of 2008 international conference on reconfigurable computing and FPGAs, vol 2009, pp 385–390

Wu Y, Zhou Y, Saveriades G, Agaian S, Noonan JP, Natarajan P (2013) Local Shannon entropy measure with statistical tests for image randomness. Inf Sci (N Y) 222:323–342